

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #10

Graphs

Computer Science

Agenda

- **Graphs**
 - Representations
 - Search for a path
 - One way
 - V1
 - V2
 - V3 – nonmonotonic reasoning!
 - Best way
 - All ways
 - Restricted way

Graphs - Representations

- **Traditional representations:**
 - Adjacency matrix – not efficient here
 - Adjacency lists – 2 ways of doing this:
 - **Neighbor list:**
 - Knowledge base with facts of
 - (node, list of neighbors)
 - **Adjacency list**
 - Knowledge base with facts of edge pairs:
 - (node, node)

Graphs - Representations

- Neighbor list – Knowledge base with facts of nodes, list of neighbors

```
% neighb/2 (+Vertex, +NeighborList) .  
neighb(a, [b, c]) .
```

- Directed graphs? What changes?

- Isolated nodes?

```
neighb(z, []) .
```

Graphs - Representations

- Adjacency list - Knowledge base with edge pairs

```
% edge/2 (+Vertex1, +Vertex2).  
edge(a,b).  
edge(a,c).
```

- UNdirected graphs? Add a predicate to ask for searching both ways.

```
% is_edge/2 (+Vertex1, +Vertex2).  
is_edge(X,Y):-  
    edge(X,Y); % check one way  
    edge(Y,X). % and the other way
```

- Isolated nodes?

```
edge(z,nil). % not a built-in way; just assume an edge to some dummy vertex
```

Search for a path in a graph

- Finding the way out from the labyrinth
- Labyrinth =
 - a set of rooms
 - With doors linking them
 - Unique entrance
 - One objective = way out = reach a given room
 - You **know** you reach it just **AFTER** you entered the room! (that is, you cannot define the problem in terms of "go to room x " as you don't know what x would be beforehand. You have this ability just after you reach there).

Computer Science

Search for a path in a graph

Labyrinth representation

- Is a graph
 - Rooms = vertices
 - Doors = edges
- Is it directed? Why/why not?
 - Graph undirected => add necessary predicate
- Enter the labyrinth from room *a*.

```
is_door(a,b).  
is_door(b,c).  
is_door(b,e).  
is_door(c,d).  
is_door(d,e).  
is_door(e,f).  
is_door(e,g).
```

```
is_objective(g).
```

```
is_pass(X,Y):-  
    is_door(X,Y);  
    is_door(Y,X).
```

Search for a path in a graph

Labyrinth way out

- Search the way out = check every possible root. When deadlock, backtrack. Prolog advantage: backtrack is there for free.

```
% path /3 (+Source, +Target, -Path).
```

```
path(X, Y, Path) :-  
    try(X, Y, [X], Path),           % try a path from X to Y with the PartialPath  
                                     % containing just the starting vertex at first  
    is_objective(Y), !.             % why not start with this?
```

Call it with:

```
?- path(a, X, Path).                % is X safe here? Not Y?
```


Search for a path in a graph

Labyrinth way out – main predicate (v1)

```
% try1 /4 (+Source, +Target, +PartialPath, -FinalPath)
try1(Y,Y,FPath,FPath).    % at every step, stop and check if over
try1(X,Y,PPath,FPath):-  % if not over (how do we know is not over here?)
    is_pass(X,Z),          % find next step to Z
    not(member(Z,PPath)), % verify if unchecked door
    try1(Z,Y,[Z|PPath],FPath). % make the step
```

- Order of clauses? Why?
- Should it be `is_pass(X,Z)`? Why not `is_door(X,Y)`?
- Pattern composition `[Z|PPath]` in the recursive call?
 - Is it correct? Shouldn't it be in the head of the rule? Why?
- Make the analysis of the calling tree.
 - When does try stop? When does try eventually close?
- What answer would we get to the initial query? Discussion on `Path`.

```
?- path(a,X,Path).
```

Search for a path in a graph

Labyrinth way out – main predicate (v2)

```
% try2/4 (+Source, +Target, +PartialPath, -FinalPath)
try2(Y,Y,_,[Y]).
try2(X,Y,PPath,[X|FPath]):-
    is_pass(X,Z),
    not(member(Z,PPath)),
    try2(Z,Y,[Z|PPath],FPath).
```

- Initial call?
- Why do we need both `PPath` and `FPath`? Do we really need both?
- Just use the 4th arg?
 - With `not(member(Z,PPath))`?
- Is it possible? What do `PPath` and `Path` contain? Make the difference
 - Understand and NEVER make confusions. Learn on the meaning of pattern composition on:
 - The head of the rule
 - Recursive call

Search for a path in a graph

Labyrinth way out – main predicate (v3)

```
% try /3 (+Source, +Target, -Path).
try3(Y,Y,[Y]).
try3(X,Y,[X|Path]):-
    is_pass(X,Z),
    accept(Z),                % can Z be part of the PartialPath
    try3(Z,Y,Path).

% accept /1 (+Vertex).
accept(X):-
    seen(X),!,
    fail.
accept(X):-
    assert(seen(X)).
accept(X):-
    retract(seen(X)),!,
    fail.
```

Search for a path in a graph

Labyrinth way out – main predicate (v3)

- Is the predicate which states if and WHEN a vertex makes part of the solution
- Reasoning is nonmonotonic (part of default logic)
 - During the reasoning,
 - If an assumption does NOT already take part of the reasoning
 - And does NOT contradict ANY other assumption
 - Is added to the knowledge base
 - If at a latter point
 - If the reasoning cannot be closed (completed), chance is made to attempt some reasoning WITHOUT that piece of knowledge
 - Therefore, quantity of knowledge is not monotonic

```
accept(X) :-
    seen(X) , ! ,      % is the contradiction! The ONLY contradiction is that the vertex is
    fail.              % ALREADY in the solution. If there, don't loop; fail to backtrack!

accept(X) :-
    assert(seen(X)) .   % no contradiction, add it in the solution

accept(X) :-
    retract(seen(X)) , ! , % cannot conclude with x in solution, remove and
    fail.                % backtrack to try WITHOUT it!
```

Labyrinth way out

v3 / v2 comparative analysis

```
% Code v3
accept (X) :-
    seen (X) , ! ,
    fail.

accept (X) :-
    assert (seen (X) ) .
    % not (member (Z , Thread) ) succeeds, hence
    % Z could be added to the current attempted solution

accept (X) :-
    retract (seen (X) ) , ! ,
    fail.
    % is there any point in v2 where we do this?
    % if so, where?
    % if not, are solutions similar?

% v3 comments
% although x was in the solution
% at some point since there is no final way
% decide to remove it from path
```

Graphs – Find the Best way

```
best_path(X,Y,_):-
    assert(best([],1000)), % not a wise init; better sum of all weights
    a_path(X,Y,[X],1).
best_path(_,_,Thread):-
    retract(best(Thread,_)).

a_path(Y,Y,Path,PathLen):-
    is_objective(Y), !,
    retract(best(_,_)), !,
    asserta(best(Path,PathLen)),
    fail. % cut above. So, where does backtracking fail? Mistake?
a_path(X,Y,Path,PathLen):-
    best(_,BestLen),
    PathLen1 is PathLen +1,
    PathLen1 < BestLen,
    is_pass(X,Z), % nondeterministic call. Why nondeterministic?
    not(member(Z,Path)),
    a_path(Z,Y,[Z|Path],PathLen1).
```

Graphs – Find all paths

- Assume neighbor representation:

```
% a_path /3 (+Source, +Target, -Path).
```

```
a_path(Y,Y,[Y]).
```

```
a_path(X,Y,[X|Rest]):-
```

```
    neighb1(X,L),
```

```
    all_paths(L,Y,Rest).
```

```
% all_paths /3 (+SourcesList, +Target, -Path).
```

```
all_paths([X|_],Y,Path):-
```

```
    a_path(X,Y,Path).
```

```
all_paths([_|Rest],Y,Path):-
```

```
    all_paths(Rest,Y,Path).
```

```
% Call it with:
```

```
% ?- all_paths([a], X, Path).
```

- Q: How are loops avoided?
- How/when does this work?

Graphs – Find restricted path

- Assume again the edge representation.

```
% restricted_path /4 (+Source, +Target, +RestrictionsList, -Path)
restricted_path(X,Y,Restrictions,Path):-
    path_obj(X,Y,Path),
    restrict(Restrictions,Path). % how else could we do?

path_obj(X,Y,Path):-
    nonvar(X), % meaning? Why needed?
    try2(X,Y,[X],Path), % any try from v1, v2, v3
    is_objective(Y).

restrict([H|TR],[H|T]):- !, % what is this cut cutting?
    restrict(TR,T).
restrict([HR|TR],[_|T]):-
    restrict([HR|TR],T).
restrict([],_).
```


Back to graphs

The issue with this implementation

```
% path1 /3 (+Source, +Target, -Path)
path1(X, Y, Path):- path1(X, Y, [X], Path).

path1(Y, Y, _, []).
path1(X, Y, PPath, [Z|FPath]):-
    edge(X, Z),
    not(member(Z, PPath)),
    path1(Z, Y, [Z|PPath], FPath).
```

- The graph is represented as a set of facts `edge/2`, if we want to run this on another graph, we have to change the code

```
?- path1(a, g, Path).
```

Back to graphs

Use = ..

```
% path1 /4 (+Graph, +Source, +Target, -Path)
path1(Graph, X, Y, Path):-
    path1(Graph, X, Y, [X], Path).

path1(_, Y, Y, _, []).
path1(G, X, Y, PPath, [Z|FPath]):-
    Edge=..[G, X, Z], % G= edge from query,
                      % Edge = G(X, Z) = edge(X, Z)
    call(Edge),
    not(member(Z, PPath)),
    path1(G, Z, Y, [Z|PPath], FPath).

path1(edge, a, g, Path).
```